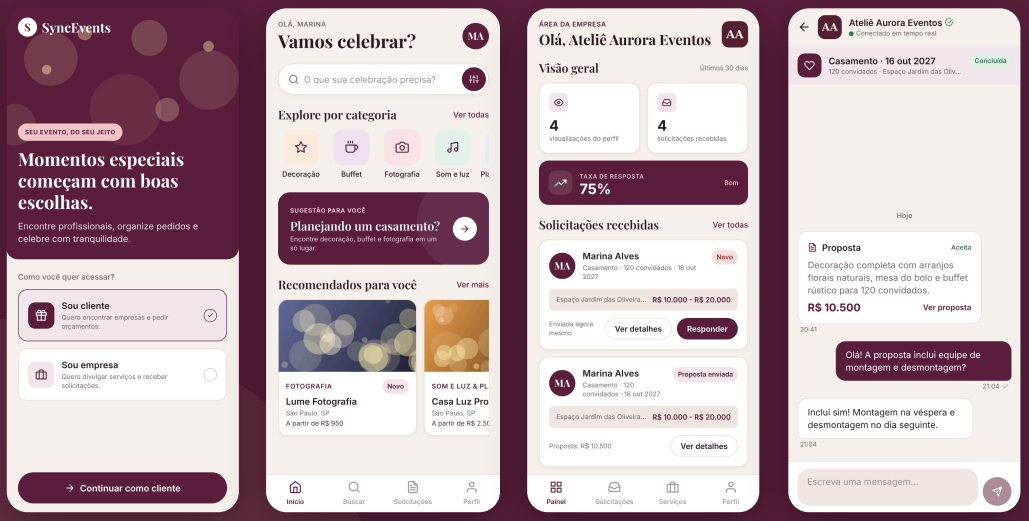
SyncEvents

DOCUMENTAÇÃO DO PROJETO

Momentos especiais começam com boas escolhas.

App Android que conecta clientes a empresas organizadoras de eventos: busca, orçamentos, propostas e chat em tempo real.



Estado: fases 0 a 5 concluídas · fases 6 (avaliações) e 7 (acabamento) pendentes

Stack: React Native (Expo SDK 58) · Java 21 + Spring Boot 3.5 · PostgreSQL 16

Repositório: github.com/PedroLuizR/SyncEvents

Gerado em: 12 de novembro de 2026

Sumário

6 de 8
fases concluídas

295
testes automatizados

60
decisões registradas

1 Visão geral

2 Arquitetura

3 Telas

4 API

5 Testes

6 Registro de decisões

7 Anexo: plano de implementação

Visão geral

App Android que conecta **clientes** que estão organizando um evento a **empresas organizadoras e prestadores de serviço**: o cliente encontra empresas, pede orçamento, conversa em tempo real e recebe propostas; a empresa divulga seus serviços, responde pedidos e acompanha seus indicadores.

Mobile	React Native (Expo SDK 58) · TypeScript · Expo Router · TanStack Query · Zustand · react-hook-form + zod · STOMP
Backend	Java 21 · Spring Boot 3.5 (Web, Security, Data JPA, WebSocket) · JWT · PostgreSQL 16 · Flyway
Testes	JUnit 5 · Mockito · Testcontainers · MockMvc · Jest · React Native Testing Library · Maestro

Status

Fase	Entrega	Histórias	Status
0	Fundação: monorepo, Docker, backend base, app base e design system	—	✅
1	Boas-vindas, login (e-mail ou CNPJ) e cadastro de cliente e de empresa	ID01, ID02, ID03	✅
2	Perfil da empresa (logo, capa, portfólio) e serviços (rascunho, ativo/inativo)	ID04, ID05	✅
3	Início do cliente, busca com filtros, perfil público da empresa, dados de demonstração	ID06, ID07	✅
4	Pedido de orçamento, propostas, gestão pela empresa, painel com indicadores	ID08, ID10	✅
5	Chat em tempo real (WebSocket/STOMP) com propostas na conversa	ID09	✅
6	Avaliações (nota, destaques, média no perfil)	ID11	⏳
7	Acabamento: revisão visual, acessibilidade, fluxos Maestro completos, APK de release	—	⏳

Números atuais: **149 testes no backend** (93 unitários + 56 de integração com Postgres real) e **146 no mobile**, todos passando, com typecheck, lint e Prettier sem erros.

Funcionalidades

Cliente

- Cadastro em 2 etapas (CPF e foto opcionais) e login; sessão salva com segurança (SecureStore).
- Início com busca, categorias, banner e empresas recomendadas para a sua cidade.
- Busca por nome, serviço ou categoria (sem acento), com filtro por categoria e cidade, ordenação e rolagem infinita.
- Perfil da empresa com portfólio, serviços e preços, indicadores, WhatsApp e compartilhar.
- Pedido de orçamento em 3 etapas (dados → revisão → enviado), com fotos de inspiração.
- "Minhas solicitações" por status; aceitar ou recusar propostas; cancelar o pedido.
- Chat em tempo real com cada empresa, com a proposta aparecendo como card na conversa.

Empresa

- Cadastro em 2 etapas com código público de 6 dígitos; login por e-mail ou CNPJ.
- Perfil editável (logo, capa, contatos, endereço, categorias e portfólio de até 12 fotos).
- Serviços com fotos, modelos de preço (fixo, a partir de, faixa, sob consulta), itens inclusos, rascunho e ativo/inativo.
- Painel com visualizações do perfil, solicitações e taxa de resposta dos últimos 30 dias (com variação).
- Gestão das solicitações: em análise, proposta, recusa e conclusão (máquina de estados validada no backend).

- Chat em tempo real com cada cliente.

Como rodar

Pré-requisitos: Docker · Java 21 + Maven 3.9 · Node 20+ · Android Studio com emulador (ou aparelho).

```
# 1. Banco de dados
cp .env.example .env
docker compose up -d postgres

# 2. Backend (perfil dev: migrations + dados de demonstração na primeira vez)
cd backend
mvn spring-boot:run

# 3. App (em outro terminal)
cd mobile
cp .env.example .env
npm install
npx expo run:android      # ou: npx expo start --web  (só para ver a interface no navegador)
```

Endereço	O quê
http://localhost:8080/api/v1	API REST
ws://localhost:8080/ws	WebSocket (STOMP) do chat
http://localhost:8080/swagger-ui.html	Documentação interativa da API
http://localhost:8080/actuator/health	Saúde do backend

- `EXPO_PUBLIC_API_URL` (em `mobile/.env`): `http://10.0.2.2:8080` no emulador Android; num aparelho físico, o IP da sua máquina.
- Variáveis do backend em `backend/.env.example`. Fora do perfil `dev`, `APP_JWT_SECRET` (32+ caracteres) é obrigatório.
- pgAdmin opcional: `docker compose --profile tools up -d` → http://localhost:5050.

Contas de demonstração (perfil `dev`)

Perfil	Login	Senha
Cliente	`marina@demo.syncevents.com`	`Senha@123`
Empresa	`contato@aurora.demo.syncevents.com`	`Senha@123`

São criadas 10 empresas com logo, capa, portfólio e serviços (lista em `DemoSeeder.java`). Para desligar: `APP_SEED_DEMO=false`.

Testes

```
cd backend && mvn verify                      # unitários + integração (Testcontainers; precisa do Docker)
TEST_DB_URL=jdbc:postgresql://localhost:5432/syncevents_test mvn verify   # sem Docker

cd mobile
npm test && npm run typecheck && npm run lint && npx prettier --check .
maestro test e2e/                             # E2E com o app instalado no emulador
```

O que cada suíte cobre: `docs/testes.md`.

Documentação

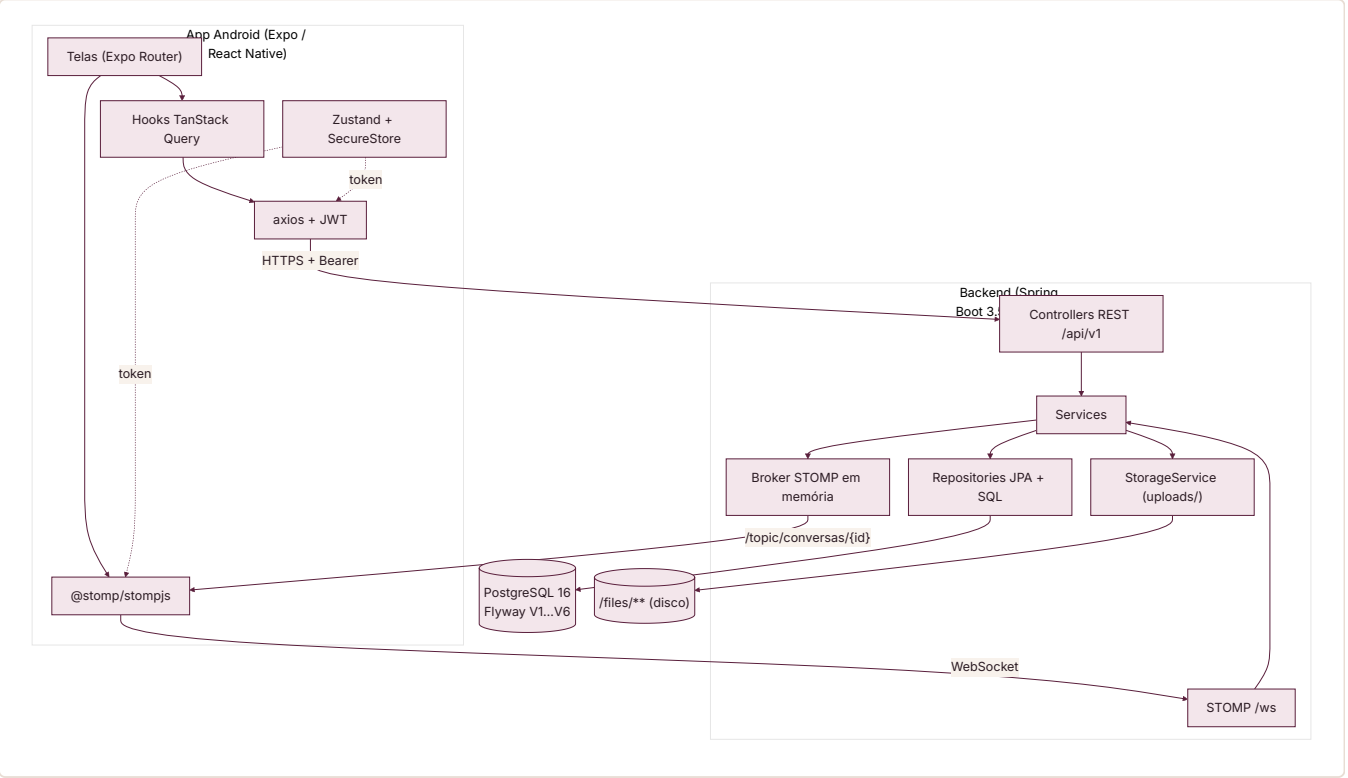
Documento	Conteúdo
`docs/arquitetura.md`	Visão geral, pacotes, telas, modelo de dados, fluxos de autenticação, orçamento e chat (com diagramas)
`docs/api.md`	Contrato da API REST e do WebSocket, fase a fase, com exemplos e regras
`docs/decisoes.md`	Todas as decisões tomadas e diferenças em relação ao Figma (60 itens)
`docs/testes.md`	Estratégia de testes e o que cada suíte verifica
`docs/telas/`	Prints de cada fase rodando contra o backend real
`SYNCEVENTS_PLANO.md`	Plano original de implementação (produto, regras, fases)

Estrutura

```
syncevents/
├── docker-compose.yml        PostgreSQL 16 (+ pgAdmin opcional)
├── backend/                  Spring Boot, pacote por domínio em br.com.syncevents.*
│   └── src/main/resources/db/migration   V1…V6 (Flyway)
├── mobile/                   Expo
│   ├── src/app/              rotas (Expo Router): (auth), cliente, empresa, chat, conversas
│   ├── src/components/       design system
│   ├── src/api · hooks/      cliente HTTP e hooks do TanStack Query
│   ├── src/theme/            tokens (cores, tipografia, espaçamentos)
│   └── e2e/                  fluxos Maestro por fase
└── docs/                     arquitetura, API, decisões, testes, design e prints
```

Arquitetura

Visão geral



- **Monorepo** com `backend/` e `mobile/`; o banco roda no Docker (`docker-compose.yml`).
- O app conversa com o backend por **REST** (dados e ações) e por **WebSocket/STOMP** (chat em tempo real).
- Uploads (logos, capas, portfólio, fotos de serviço e inspirações) ficam em disco e são servidos em `/files/**`, atrás da interface `StorageService` (pode virar S3/MinIO sem mudar quem usa).

Backend

Pacote por domínio, sempre `Controller → Service → Repository`, DTOs em `record` separados das entidades e `@Transactional` nos services.

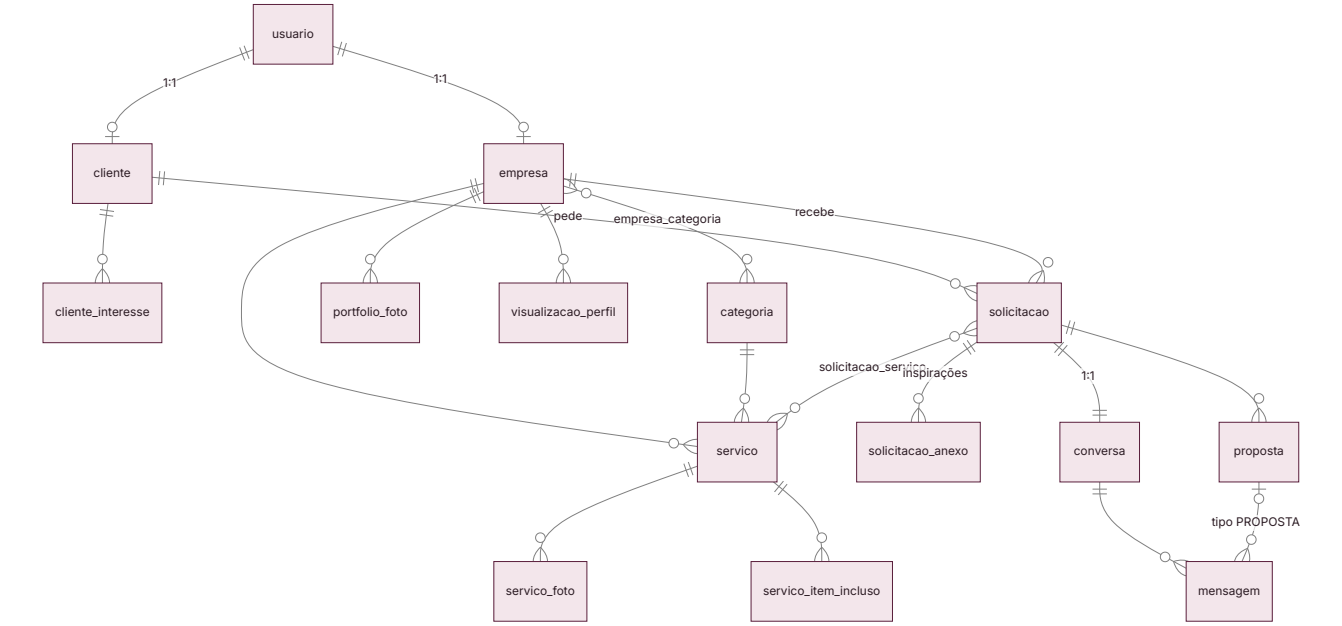
Pacote	Responsabilidade
`config`	Security (stateless + JWT), CORS, OpenAPI, WebSocket/STOMP, propriedades `app.*`
`auth`	Cadastro de cliente e empresa, login, `/auth/me`, geração e validação de JWT
`usuario`, `cliente`, `empresa`	Entidades de conta e perfil; perfil e portfólio da empresa; visualizações
`categoria`	Categorias de serviço (seed)
`servico`	CRUD de serviços com fotos, itens inclusos, rascunho e ativo/inativo
`busca`	Busca textual sem acento (SQL + `unaccent`), recomendados e perfil público
`solicitacao`	Pedidos de orçamento, máquina de estados, propostas e painel da empresa
`chat`	Conversas, mensagens, histórico paginado, lidas e publicação no broker
`storage`	Gravação e remoção de imagens

Pacote	Responsabilidade
`seed`	Dados de demonstração (perfil `dev`)
`common`	Formato de erro único (`ApiErro`), handler global, validadores (senha, CNPJ, telefone, nome)

Segurança

- Senhas com **BCrypt**; **JWT HS256** com validade de 7 dias (`sub` = id do usuário, `tipo` = CLIENTE/EMPRESA).
- O filtro `JwtAuthFilter` transforma o token em `ROLE_CLIENTE` ou `ROLE_EMPRESA`; as rotas exclusivas de cada perfil são protegidas no `SecurityConfig`.
- **Posse e participação** são checadas nos services: outra empresa editando um serviço recebe 403, quem não participa de uma solicitação ou conversa recebe 403.
- Todas as respostas de erro seguem o mesmo formato: `{ status, erro, mensagem, campos }`.

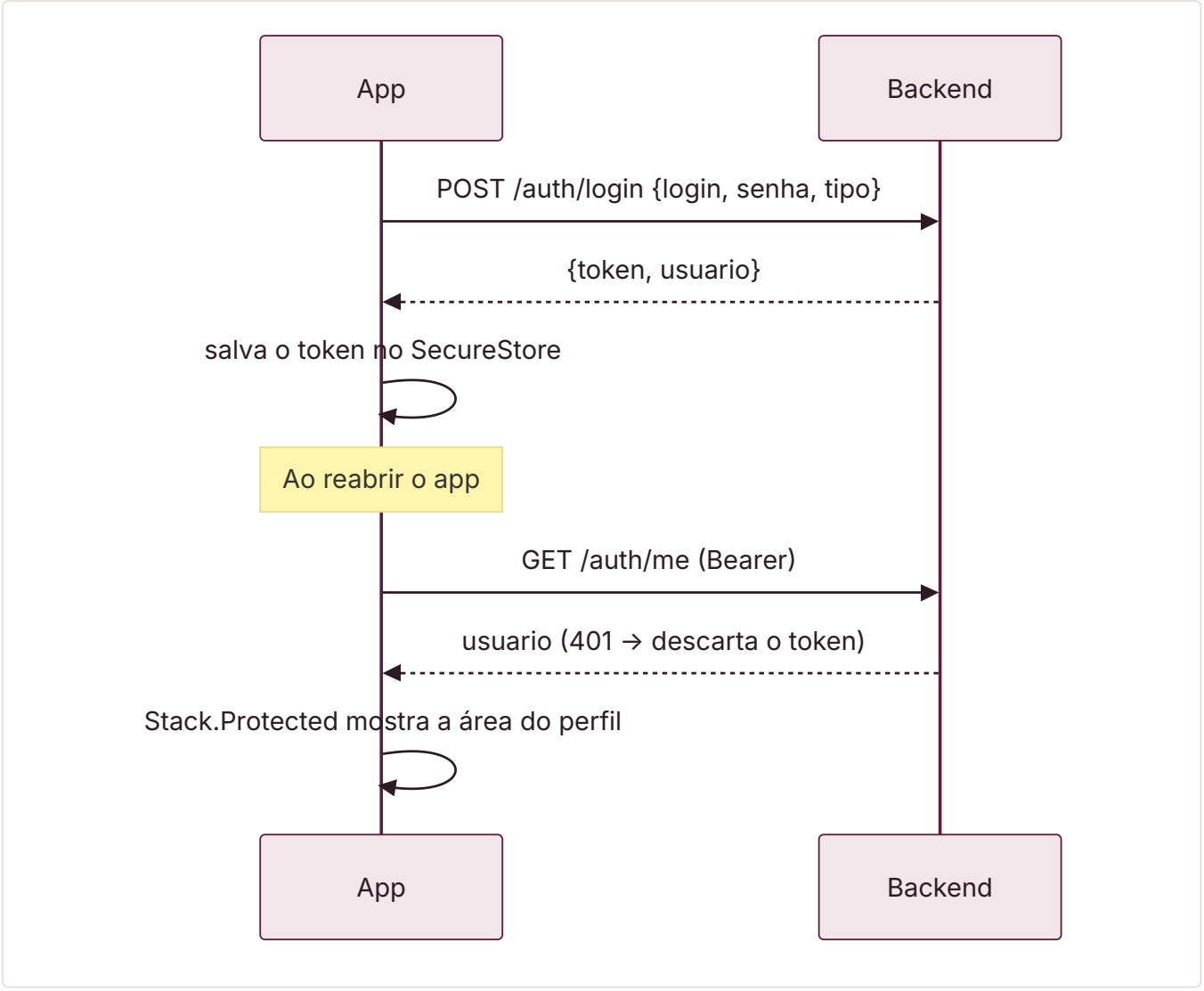
Modelo de dados



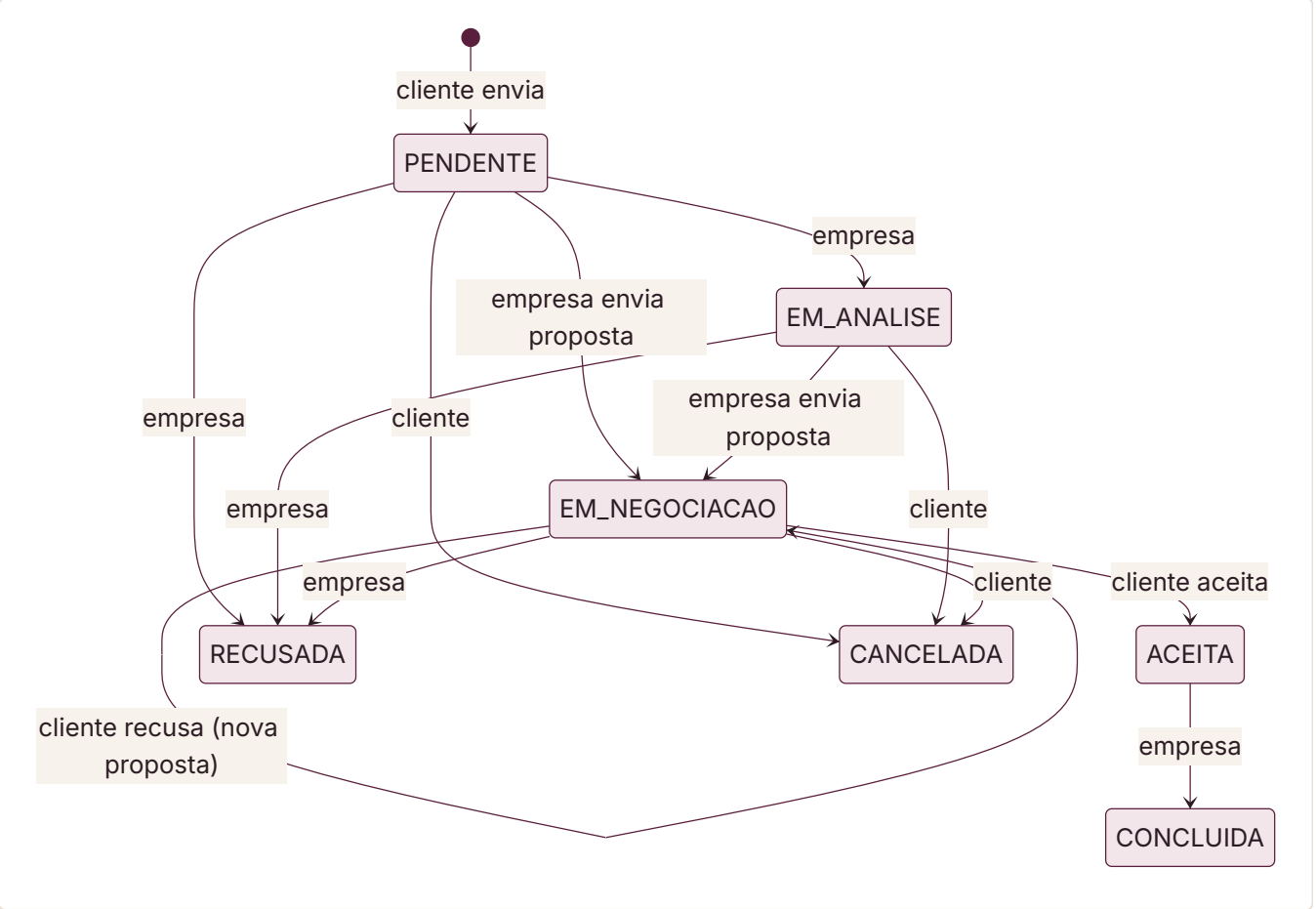
Migrations Flyway: `V1` (unaccent) · `V2` usuários, clientes, empresas e categorias · `V3` serviços e portfólio · `V4` visualizações · `V5` solicitações, propostas e conversas · `V6` mensagens. A tabela `avaliacao` chega na Fase 6.

Fluxos

Autenticação

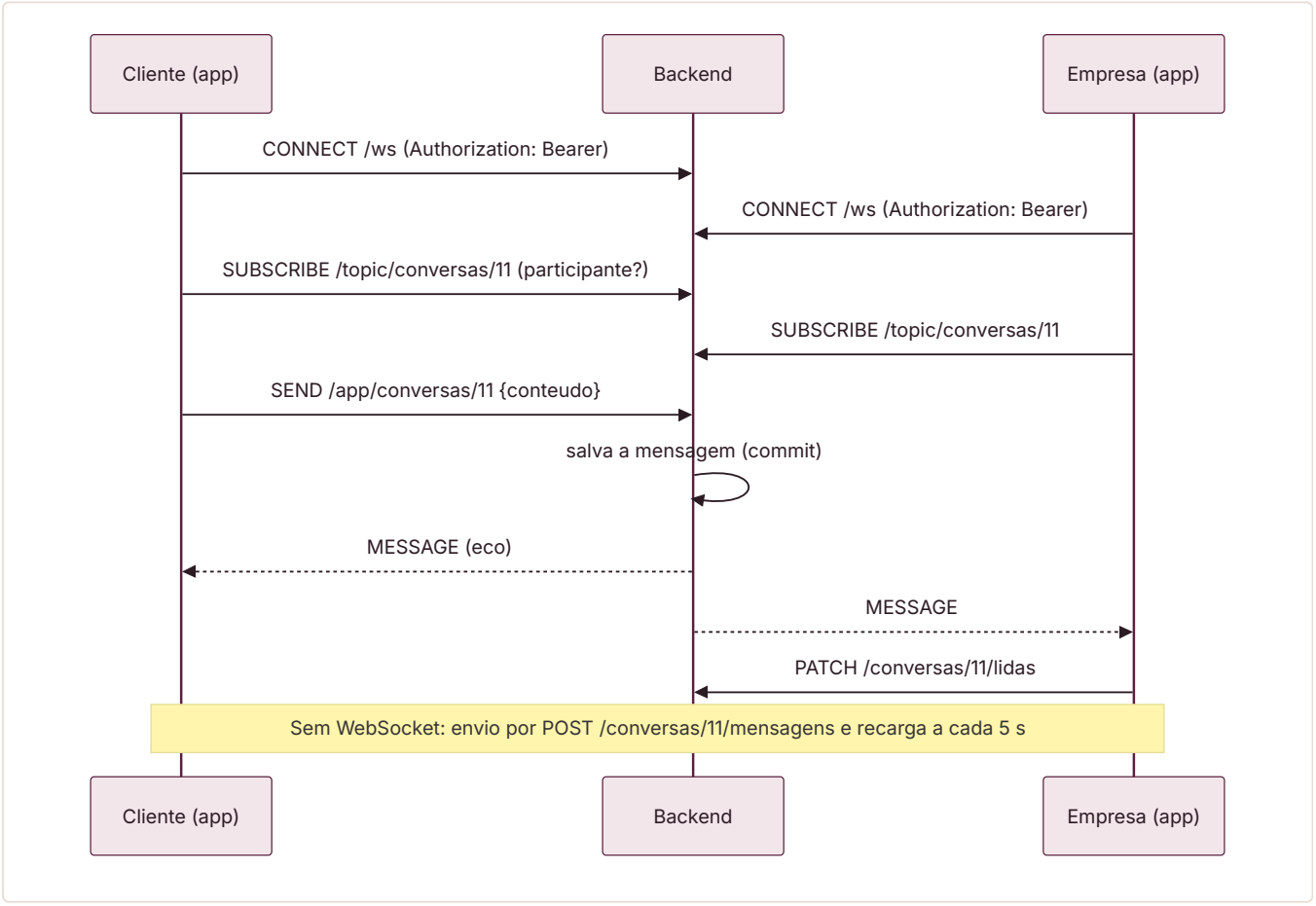


Solicitação de orçamento



Qualquer transição fora do diagrama responde **409**. Ao criar a solicitação, a **conversa** é criada junto.

Chat em tempo real



Propostas enviadas pela empresa também viram mensagens no tópico; quando o cliente aceita ou recusa, a mesma mensagem é publicada de novo com o status novo.

Mobile

Pasta	Conteúdo
`src/app/`	Rotas do Expo Router. `(auth)` público; `cliente/` e `empresa/` com abas próprias; `chat/` e `conversas` compartilhados. O acesso é controlado por `Stack.Protected` no layout raiz.
`src/components/`	Design system (Button, TextInput, Chip, Card, Select, SwitchCard, EmpresaCard, SolicitacaoCard, BolhaMensagem...)
`src/theme/`	Tokens de cor, tipografia (Playfair Display + Inter), espaçamentos e raios: nenhum componente usa cor fixa
`src/api/`	Cliente axios (injeta o JWT, trata 401) e funções por recurso
`src/hooks/`	Hooks do TanStack Query: as telas só falam com a API por eles
`src/chat/`	Conexão STOMP (reconexão automática)
`src/schemas/`	Validações zod, com as mesmas regras e mensagens do backend
`src/store/`	Sessão (Zustand + SecureStore)
`src/utils/`	Máscaras (telefone, CNPJ, CPF, moeda), formatação de datas e preços, helpers do chat

Telas por perfil

Cliente	Empresa	Ambos
Início · Buscar · Minhas solicitações · Perfil (abas)	Painel · Solicitações · Serviços · Perfil (abas)	Boas-vindas · Login · Cadastros
Perfil da empresa · Pedido de orçamento · Detalhe da solicitação	Criar/editar serviço · Editar perfil · Detalhe da solicitação	Conversas · Chat

Telas

Prints de cada fase, gerados com o app (versão web) rodando contra o backend e o banco reais.

Fase 0: Fundação e design system



Catálogo do design system

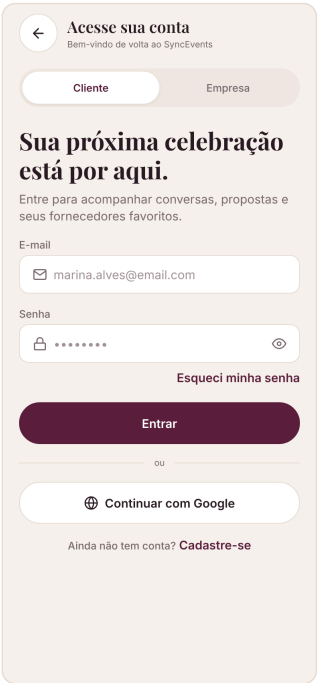


Catálogo: cards, avaliação e estado vazio

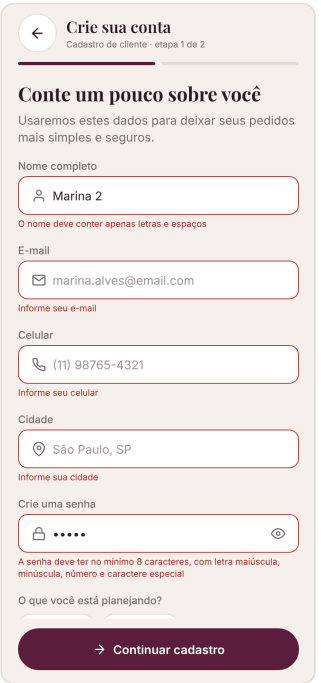
Fase 1: Autenticação e cadastros



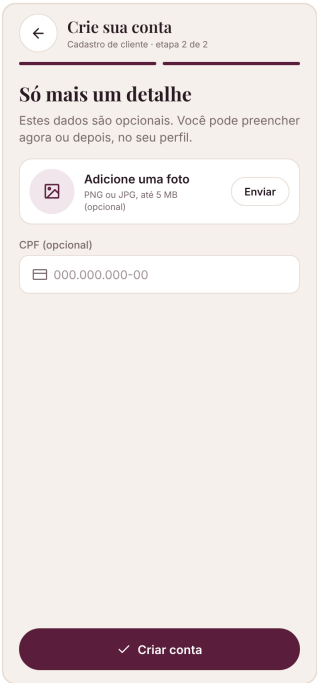
Boas-vindas: escolha do perfil



Login (abas Cliente/Empresa)



Cadastro de cliente: validações



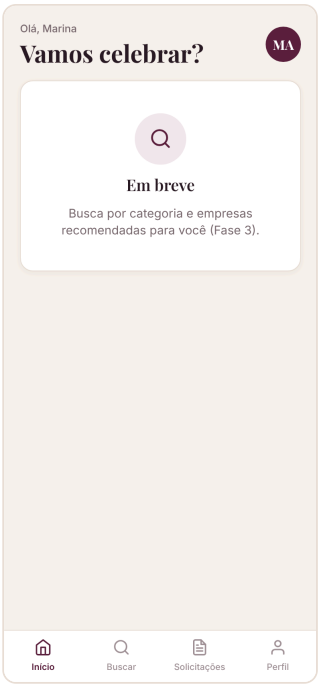
Cadastro de cliente: etapa 2 (CPF e foto opcionais)



Volta ao login após o cadastro



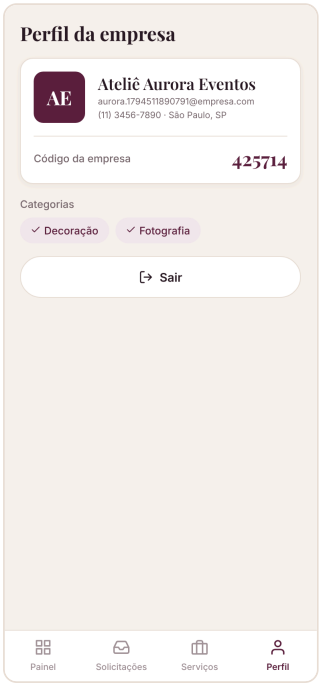
Login com senha errada



Primeira versão do Início



Cadastro de empresa: etapa 2



Perfil da empresa com o código de 6 dígitos

Fase 2: Perfil da empresa e serviços



Novo serviço: validações



Novo serviço preenchido



Meus serviços (ativo e rascunho)



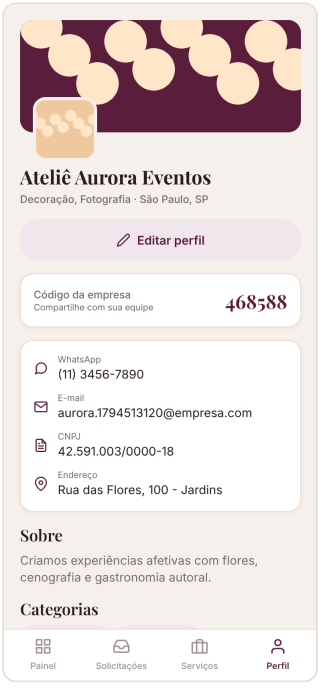
Editar serviço



Serviço desativado

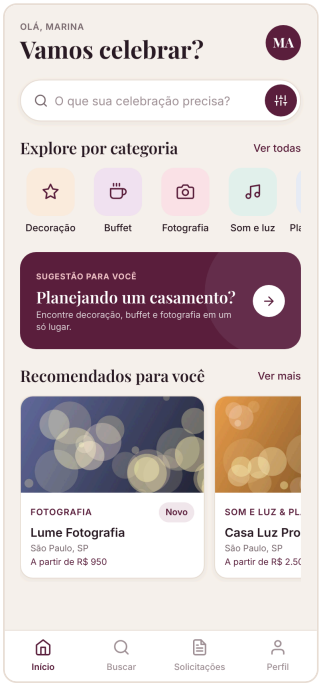


Editar perfil da empresa



Perfil com capa, logo e contatos

Fase 3: Descoberta do cliente



Início do cliente



Busca por "decoração"



Filtros de cidade e ordenação



Resultados filtrados por cidade e preço



Perfil da empresa visto pelo cliente

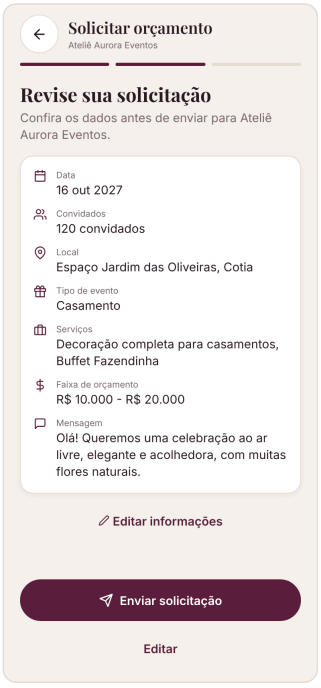


Perfil: portfólio, serviços e avaliações

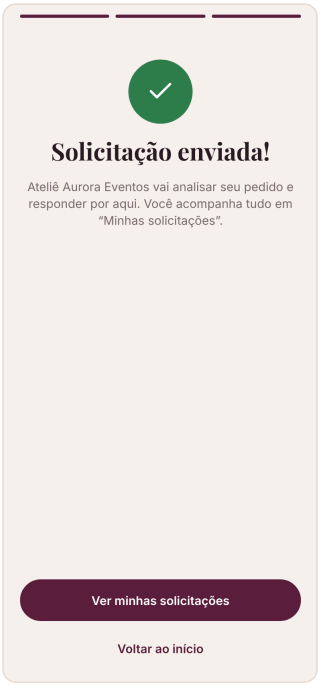
Fase 4: Orçamentos e painel



Solicitar orçamento: dados



Solicitar orçamento: revisão



Solicitação enviada



Minhas solicitações



Painel da empresa



Empresa enviando proposta

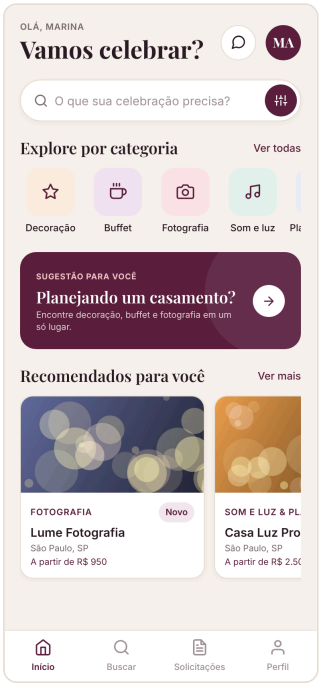


Proposta vista pela cliente

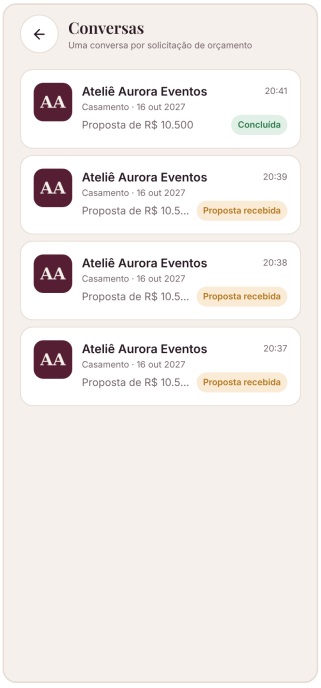


Evento concluído

Fase 5: Chat em tempo real



Início com o ícone de conversas



Lista de conversas



Chat visto pela cliente



Chat visto pela empresa

API

- Base: `http://localhost:8080/api/v1` (no emulador Android: `http://10.0.2.2:8080/api/v1`)
- Documentação interativa: `/swagger-ui.html` · OpenAPI em JSON: `/v3/api-docs`
- Autenticação: `Authorization: Bearer <jwt>` (a partir da Fase 1)

Formato de erro

Todas as falhas respondem neste formato (`campos` só aparece em erros de validação):

```
{ "status": 400, "erro": "VALIDACAO", "mensagem": "Dados inválidos", "campos": { "email": "E-mail já cadastrado" } }
```

HTTP	`erro`	Quando
400	`VALIDACAO`	Bean Validation ou regra de campo (ex.: e-mail duplicado)
400	`REQUISICAO_INVALIDA`	JSON mal formado, parâmetro ausente ou com tipo errado
401	`NAO_AUTENTICADO`	Rota protegida sem token válido
403	`ACESSO_NEGADO`	Usuário sem permissão sobre o recurso
404	`NAO_ENCONTRADO`	Recurso ou rota inexistente
405	`METODO_NAO_PERMITIDO`	Método HTTP não suportado na rota
409	`CONFLITO`	Conflito de estado (ex.: transição de status inválida)
413	`ARQUIVO_GRANDE`	Upload acima de 10 MB por arquivo / 30 MB por requisição
500	`ERRO_INTERNO`	Erro inesperado (detalhes só no log)

Rotas públicas (sem token)

`/actuator/health`, `/swagger-ui/**`, `/v3/api-docs/**`, `/files/**`, `POST /api/v1/auth/cadastro/**`, `POST /api/v1/auth/login` e `GET /api/v1/categorias/**`. Qualquer outra rota exige autenticação.

Fase 1: autenticação e cadastros

`GET /categorias` (público)

```
[{ "id": 1, "nome": "Decoração", "icone": "star" }, { "id": 2, "nome": "Buffet", "icone": "coffee" }, ...]
```

Seed: Decoração, Buffet, Fotografia, Som e luz, Planejamento, Bebidas. `icone` é o nome de um ícone Feather.

`POST /auth/cadastro/cliente` (público, `multipart/form-data`) — ID01

Campo	Obrigatório	Regra
`nome`	sim	só letras (com acento) e espaços, até 120
`email`	sim	formato válido e único (sem diferenciar maiúsculas)
`telefone`	sim	`(00) 00000-0000` ou só dígitos (10 ou 11)

Campo	Obrigatório	Regra
`cidade`	sim	texto livre, ex.: `São Paulo, SP`
`senha`	sim	8+ caracteres com maiúscula, minúscula, número e especial
`interesses`	não	repetir o campo: `CASAMENTO`, `ANIVERSARIO`, `CORPORATIVO`, `FORMATURA`, `INFANTIL`, `OUTRO`
`aceiteTermos`	sim	`true`
`cpf`	não	texto livre, **sem validação**
`foto`	não	arquivo PNG, JPG ou WEBP, até 5 MB

Resposta `201` com o `Usuario` (abaixo). Não devolve token: o app volta para o login.

`POST /auth/cadastro/empresa` (público, `multipart/form-data`) — ID02

Campo	Obrigatório	Regra
`nomeFantasia`	sim	até 120
`cnpj`	sim	14 dígitos, com ou sem máscara (dígitos verificadores **não** são conferidos); único
`email`	sim	formato válido e único
`telefone`	sim	WhatsApp da empresa
`cidade`	sim	se terminar em `, UF`, a UF é guardada separadamente
`categoriaIds`	sim	repetir o campo; ids existentes
`descricao`	não	até 2000
`endereco`	não	texto livre, **sem verificação**, editável depois
`senha`	sim	mesma regra do cliente
`aceiteTermos`	sim	`true`
`logo`	não	PNG, JPG ou WEBP, até 5 MB

Gera o `codigo` público de 6 dígitos (aleatório e único). Resposta `201` com o `Usuario`.

`POST /auth/login` (público) — ID03

```
{ "login": "marina@email.com", "senha": "Senha@123", "tipo": "CLIENTE" }
```

Com `tipo: "EMPRESA"`, `login` aceita e-mail **ou CNPJ** (com ou sem máscara).

```
{ "token": "<jwt>", "usuario": { ...Usuario } }
```

Situação	Resposta
Senha errada ou usuário inexistente	`401 CREDENCIAIS_INVALIDAS` "E-mail ou senha incorretos" (empresa: "E-mail/CNPJ ou senha incorretos")
Conta de outro perfil (senha certa)	`401` "Esta conta é de cliente. Entre pela aba Cliente."

O JWT (HS256) vale 7 dias (`APP_JWT_VALIDADE`) e leva `sub` (id do usuário), `email` e `tipo`.

`GET /auth/me` (autenticado)

Devolve o `Usuario` logado.

`Usuario`

```
{
  "id": 1, "email": "marina@email.com", "tipo": "CLIENTE", "perfilId": 1,
  "nome": "Marina Alves", "telefone": "11987654321", "cidade": "São Paulo, SP",
  "fotoUrl": "/files/clientes/<uuid>.png",
  "interesses": ["CASAMENTO"],          // só cliente
  "codigo": "504731",                    // só empresa
  "categorias": [{ "id": 1, "nome": "Decoração", "icone": "star" }]  // só empresa
}
```

`fotoUrl` é a foto do cliente ou a logo da empresa, servida pelo backend em `/files/**`.

Fase 2: empresa: perfil e serviços

Todas exigem token de **empresa** (cliente recebe `403`). A posse é checada no backend: outra empresa recebe `403` `"Este serviço pertence a outra empresa"`.

Perfil (ID04)

Método	Rota	Corpo	Resposta
GET	`/empresas/me`	—	`EmpresaPerfil`
PUT	`/empresas/me`	JSON `{ nomeFantasia, telefone, cidade, endereco?, descricao?, categoriaIds[] }`	`EmpresaPerfil`
POST	`/empresas/me/logo`	multipart `arquivo`	`EmpresaPerfil` (a logo antiga é apagada)
POST	`/empresas/me/capa`	multipart `arquivo`	`EmpresaPerfil`
POST	`/empresas/me/portfolio`	multipart `arquivo` (máx. 12 fotos)	`EmpresaPerfil`
DELETE	`/empresas/me/portfolio/{fotoId}`	—	`EmpresaPerfil` (`404` se a foto não é sua)

E-mail e CNPJ **não** são editáveis.

```
{
  "id": 1, "codigo": "504731", "nomeFantasia": "Ateliê Aurora", "cnpj": "42591003000118",
  "email": "contato@aurora.com", "telefone": "1134567890", "cidade": "São Paulo, SP", "uf": "SP",
  "endereco": "Rua das Flores, 100", "descricao": "...", "logoUrl": "/files/logos/x.png",
  "capaUrl": "/files/capas/y.png", "categorias": [{ "id": 1, "nome": "Decoração", "icone": "star" }],
  "portfolio": [{ "id": 9, "url": "/files/portfolio/z.png", "ordem": 0 }]
}
```

Serviços (ID05)

Método	Rota	Quem	Descrição
GET	`/servicos/me`	empresa	todos os seus (inclui inativos e rascunhos)
POST	`/servicos/me`	empresa	cria → `201`

Método	Rota	Quem	Descrição
GET	`/servicos/{id}`	autenticado	a dona vê qualquer um; os demais só ativos e publicados (senão `404`)
PUT	`/servicos/{id}`	dona	edita
DELETE	`/servicos/{id}`	dona	remove → `204` (apaga também as fotos)
POST	`/servicos/{id}/fotos`	dona	multipart `arquivo` (máx. 6)
DELETE	`/servicos/{id}/fotos/{fotoId}`	dona	remove a foto
GET	`/empresas/{id}/servicos`	autenticado	serviços **ativos e publicados** da empresa

Corpo de criação/edição:

```
{
  "nome": "Decoração completa", "categoriaId": 1, "descricao": "Projeto personalizado...",
  "modeloPreco": "A_PARTIR_DE", "precoMin": 2800, "precoMax": null,
  "ativo": true, "rascunho": false, "itensInclusos": ["Projeto e moodboard", "Montagem"]
}
```

Regra	Mensagem
Sempre: nome e categoria	"Informe o nome do serviço", "Escolha a categoria"
Para publicar (`rascunho: false`): descrição	"Descreva o serviço"
Para publicar: modelo de preço	"Escolha o modelo de preço"
`FIXO` / `A_PARTIR_DE` / `FAIXA`: valor mínimo > 0	"Informe o valor"
`FAIXA`: valor máximo > 0 e ≥ mínimo	"Informe o valor máximo" / "O valor máximo deve ser maior ou igual ao mínimo"

Normalização: `SOB_CONSULTA` descarta os preços; `precoMax` só é guardado em `FAIXA`; itens vazios são ignorados. Um serviço aparece para clientes apenas se `ativo = true` e `rascunho = false`.

Fase 3: cliente: descoberta

`GET /empresas` (cliente) — ID06

Parâmetro	Descrição
`q`	texto livre. Ignora acentos e maiúsculas. Cada palavra (exceto "de", "para", "e"...) precisa aparecer no **nome da empresa**, numa **categoria**, no **nome de um serviço publicado** ou nas descrições
`categoriaId`	só empresas com essa categoria
`cidade`	prefixo da cidade, sem acento (ex.: `sao paulo` ou `São Paulo, SP`)
`ordenar`	`relevancia` (padrão), `nota` ou `preco` (menor "a partir de" primeiro; sem preço por último)
`page`, `size`	página a partir de 0; tamanho padrão 20, máximo 50

Relevância: nome (4) > categoria (3) > nome de serviço (2) > descrições (1), somada por palavra. Serviços inativos e rascunhos não contam.

```
{
  "conteudo": [{
    "id": 1, "nomeFantasia": "Ateliê Aurora Eventos", "logoUrl": "/files/logos/a.jpg",
    "capaUrl": "/files/capas/b.jpg", "cidade": "São Paulo, SP",
    "categorias": [{ "id": 1, "nome": "Decoração", "icone": "star" }],
    "nota": null, "totalAvaliacoes": 0, "precoAPartirDe": 2800.00
  }],
  "pagina": 0, "tamanho": 20, "total": 7, "totalPaginas": 1, "ultima": true
}
```

`GET /empresas/recomendadas` (cliente)

Até 10 empresas com ao menos um serviço publicado. As da **cidade do cliente** vêm primeiro, depois as com mais serviços.

`GET /empresas/{id}` (autenticado) — ID07

Perfil público: dados, categorias, portfólio, serviços publicados, nota e indicadores. Quando quem abre é um **cliente**, grava uma linha em `visualizacao_perfil` (usado no painel da empresa, Fase 4). Não expõe CNPJ nem e-mail.

```
{
  "id": 1, "codigo": "504731", "nomeFantasia": "Ateliê Aurora Eventos", "descricao": "...",
  "cidade": "São Paulo, SP", "uf": "SP", "endereco": "Rua Harmonia, 520", "telefoneWhatsapp": "11987654321",
  "logoUrl": "...", "capaUrl": "...", "categorias": [...], "portfolio": [...], "servicos": [...],
  "nota": null, "totalAvaliacoes": 0,
  "indicadores": { "eventosRealizados": 0, "servicos": 2, "fotos": 4, "naPlataformaDesde": 2026 }
}
```

Dados de demonstração (perfil `dev`)

Ao subir no perfil `dev`, o backend cria (uma única vez) 10 empresas com logo, capa, portfólio e serviços, e a cliente **marina@demo.syncevents.com**. Todas as contas usam a senha **Senha@123**; os e-mails das empresas terminam em `@...demo.syncevents.com` (ex.: `contato@aurora.demo.syncevents.com`). Para desligar: `APP_SEED_DEMO=false`.

Fase 4: orçamentos e gestão

Máquina de estados

```
PENDENTE ──► EM_ANALISE ──► EM_NEGOCIACAO (proposta enviada) ──► ACEITA ──► CONCLUIDA
    └──────────┴──────────────────┴──► RECUSADA   (empresa)
    └──────────┴──────────────────┴──► CANCELADA  (cliente, antes de ACEITA)
```

Ação	Quem	Rota	Permitido a partir de	Resultado
Criar	cliente	`POST /solicitacoes`	—	`PENDENTE` + conversa criada
Em análise	empresa	`PATCH /solicitacoes/{id}/status {status: EM_ANALISE}`	PENDENTE	`EM_ANALISE`
Proposta	empresa	`POST /solicitacoes/{id}/propostas {valor, descricao}`	PENDENTE, EM_ANALISE, EM_NEGOCIACAO (sem outra em aberto)	`EM_NEGOCIACAO`
Aceitar proposta	cliente	`PATCH /propostas/{id} {status: ACEITA}`	EM_NEGOCIACAO	`ACEITA`

Ação	Quem	Rota	Permitido a partir de	Resultado
Recusar proposta	cliente	`PATCH /propostas/{id}` `{status: RECUSADA}`	EM_NEGOCIACAO	continua `EM_NEGOCIACAO` (a empresa pode propor de novo)
Recusar pedido	empresa	`PATCH /solicitacoes/{id}/status` `{status: RECUSADA}`	PENDENTE, EM_ANALISE, EM_NEGOCIACAO	`RECUSADA`
Concluir	empresa	`PATCH /solicitacoes/{id}/status` `{status: CONCLUIDA}`	ACEITA	`CONCLUIDA`
Cancelar	cliente	`PATCH /solicitacoes/{id}/cancelar`	PENDENTE, EM_ANALISE, EM_NEGOCIACAO	`CANCELADA`

Qualquer outra transição responde **409** (`erro: "CONFLITO"`), por exemplo "Não é possível mudar de pendente para concluída". Quem não participa da solicitação recebe **403**.

`POST /solicitacoes` (cliente) — IDo8

```
{
  "empresaId": 7, "tipoEvento": "CASAMENTO", "dataEvento": "2027-10-16", "qtdConvidados": 120,
  "local": "Espaço Jardim das Oliveiras, Cotia", "servicoIds": [12, 13],
  "orcamentoMin": 10000, "orcamentoMax": 20000, "mensagem": "...", "aceiteContato": true
}
```

Regras: data de hoje em diante, convidados ≥ 1, serviços **publicados da própria empresa**, máximo ≥ mínimo, `aceiteContato` obrigatório. Inspirações: `POST /solicitacoes/{id}/anexos` (multipart `arquivo`, até 5).

`GET /solicitacoes/me?status=PENDENTE,EM_ANALISE`

Cliente: as suas (mais recentes primeiro). Empresa: as recebidas. Na lista, `propostaAtual` traz a última proposta.

`GET /solicitacoes/{id}`

Detalhe com `cliente` (nome, cidade, telefone, e-mail), `empresa`, `servicos`, `anexos`, `propostas` e `conversaId`.

`GET /empresas/me/dashboard` (empresa)

```
{ "dias": 30, "visualizacoes": 1248, "variacaoVisualizacoes": 18, "solicitacoes": 24,
  "variacaoSolicitacoes": 12, "taxaResposta": 92, "pendentes": 2, "servicosAtivos": 6, "servicosRascunho": 1 }
```

- **Visualizações**: aberturas do perfil por clientes nos últimos 30 dias; variação em relação aos 30 dias anteriores (`null` se o período anterior foi zero).
- **Solicitações**: recebidas nos últimos 30 dias, com a mesma comparação.
- **Taxa de resposta**: % das solicitações do período que saíram de `PENDENTE`, sem contar as canceladas pelo cliente (`null` sem solicitações).

O perfil público passa a mostrar `indicadores.eventosRealizados` (solicitações concluídas). Um serviço que aparece em alguma solicitação não pode mais ser excluído (**409**); use "desativar".

Fase 5: chat

Cada solicitação tem **uma conversa** (criada junto com o pedido). Só o cliente e a empresa envolvidos acessam: os outros recebem **403**.

REST

Método	Rota	Descrição
GET	`/conversas`	conversas do usuário (mais recente primeiro) com `contato`, contexto do orçamento, `ultimaMensagem` e `naoLidas`
GET	`/conversas/{id}`	uma conversa (cabeçalho do chat)
GET	`/conversas/{id}/mensagens?antesDe=&size=`	histórico paginado: 30 por vez (máx. 100), em ordem cronológica; `antesDe` = id da mensagem mais antiga que o app já tem; `temMais` indica se há mais antigas
POST	`/conversas/{id}/mensagens`	envia `{ "conteudo": "..." }` (fallback quando o WebSocket não está conectado)
PATCH	`/conversas/{id}/lidas`	marca como lidas as mensagens recebidas → `{ "marcadas": 3 }`

```
{
  "id": 42, "conversaId": 11, "remetenteUsuarioId": 7, "tipo": "PROPOSTA",
  "conteudo": "Pacote completo", "proposta": { "id": 90, "valor": 10500, "descricao": "...", "status":
"ENVIADA" },
  "lida": false, "enviadaEm": "2026-11-12T20:41:00Z"
}
```

WebSocket (STOMP)

- Endpoint: **`/ws`** (WebSocket puro, sem SockJS). No emulador: `ws://10.0.2.2:8080/ws`.
- `CONNECT` com o cabeçalho STOMP `Authorization: Bearer <jwt>`. Sem token válido, a conexão é recusada.
- Assinar **`/topic/conversas/{id}`** e enviar para **`/app/conversas/{id}`** com `{ "conteudo": "..." }`. Assinar ou enviar para uma conversa da qual não participa encerra a sessão.
- Tudo o que acontece na conversa é publicado no tópico como `Mensagem`: textos enviados por STOMP **ou** REST, propostas enviadas e propostas aceitas/recusadas (a mesma mensagem chega de novo com o status novo; o app substitui pelo `id`).
- Heartbeat de 10 s. O app reconecta sozinho a cada 5 s e, enquanto estiver desconectado, recarrega o histórico a cada 5 s (polling).

Erros de validação (exemplos)

```
{ "status": 400, "erro": "VALIDACAO", "mensagem": "Dados inválidos",
  "campos": { "email": "E-mail já cadastrado", "cnpj": "O CNPJ deve ter 14 dígitos" } }
```

O contrato completo planejado está na seção 7 de `SYNCEVENTS_PLANO.md`.

Testes

Cada fase entrega testes de **service** (unitários), **controller/API** (integração com MockMvc e Postgres real), **regras de autorização** e, no app, **validação de formulários e telas**. Antes de cada commit rodam: `mvn verify` no backend e `npm test`, `npm run typecheck`, `npm run lint` e `prettier --check` no mobile.

Onde	Testes	Ferramentas
Backend: unitários	93	JUnit 5, Mockito, AssertJ
Backend: integração	56	Spring Boot Test, MockMvc, Testcontainers (PostgreSQL 16), cliente STOMP real
Mobile	146	Jest (jest-expo), React Native Testing Library
E2E	6 fluxos	Maestro (`mobile/e2e/`)

Backend

Suíte	Tipo	O que garante
`ValidadoresTest`	unitário	Senha forte, CNPJ (14 dígitos), telefone e nome com acento
`GlobalExceptionHandlerTest`	unitário	Todas as falhas no formato `{status, erro, mensagem, campos}`, sem vazar detalhes internos
`JwtServiceTest`	unitário	Token válido, adulterado, de outra chave ou expirado
`AuthServiceTest`	unitário	Normalização dos dados, e-mail/CNPJ duplicado, login por CNPJ, mensagens de login errado
`CodigoEmpresaGeneratorTest`	unitário	Código de 6 dígitos único (com novas tentativas)
`ServicoServiceTest`	unitário	Regras de rascunho e publicação por modelo de preço, posse (403), limite de fotos, exclusão bloqueada
`FiltroBuscaTest`	unitário	Termos sem acento e sem palavras vazias, paginação
`StatusSolicitacaoTest`	unitário	Todas as transições válidas e inválidas da máquina de estados
`FundacaoIT`	integração	Health, Flyway, OpenAPI, CORS e 401 no formato padrão
`AuthIT`	integração	Cadastros (com foto e logo), duplicados, senha fraca, CNPJ incompleto, login, `/auth/me`
`ServicoIT`	integração	Criar, editar, desativar e remover; **outra empresa recebe 403**; cliente não gerencia serviços
`EmpresaPerfilIT`	integração	Ver e editar perfil, logo, capa e portfólio; cliente recebe 403
`BuscaIT`	integração	**"decoração" traz a categoria e quem tem serviço com esse nome**, filtros, ordenação por preço, paginação, recomendados e registro de visualização
`DemoSeederIT`	integração	Seed idempotente e utilizável pela cliente demo
`SolicitacaoIT`	integração	Fluxo completo até concluir, **transições inválidas com 409**, cancelamento, 403 para quem é de fora, validações, anexos e painel
`ChatIT`	integração	Conversa nasce com o pedido, mensagens e não lidas, proposta no chat, histórico paginado, **403 para quem é de fora**, troca **em tempo real via STOMP** entre cliente e empresa, conexão sem token recusada

Os testes de integração usam um PostgreSQL real em container (Testcontainers). Sem Docker, defina `TEST_DB_URL`.

Mobile

Suíte	O que garante
`components.test.tsx`	Button, TextInput (máscara, erro, olho da senha), Chip, RatingStars, Avatar, EmptyState, TabBar
`masks`, `validacao`, `servico`, `solicitacao`, `chat`, `whatsapp`	Máscaras, regras iguais às do backend, preços, datas, status, upsert e ordem das mensagens
`schemas/*`	Mensagens de erro de cada formulário (login, cadastros, serviço, pedido)
`login`, `cadastroCliente`, `cadastroEmpresa`	Validação, envio, erro do servidor no campo certo, volta ao login com sucesso
`servicoForm`, `meusServicos`, `editarPerfilEmpresa`	Rascunho × publicação, preço convertido, fotos, exclusão com confirmação, 403 exibido
`inicioCliente`, `buscar`, `perfilEmpresaPublico`	Navegação com filtros, rolagem infinita, estados de carregando/vazio/erro, WhatsApp
`solicitarOrcamento`, `solicitacoesCliente`, `empresaSolicitacoes`	Pedido em 3 etapas, abas por status, aceitar/recusar/cancelar, painel, proposta, conclusão, erro 409
`chat`	Histórico e card de proposta, envio por WebSocket conectado e por REST desconectado, recebimento em tempo real, aceitar no chat, lista de conversas

E2E (Maestro)

Fluxo	Arquivo
Boas-vindas	`e2e/fase0-boas-vindas.yaml`
Cadastrar cliente → logar	`e2e/fase1-cadastro-cliente-login.yaml`
Empresa cria, edita, desativa e remove serviço	`e2e/fase2-servico-empresa.yaml`
Cliente busca e abre um perfil	`e2e/fase3-descoberta-cliente.yaml`
Pedido de orçamento e proposta	`e2e/fase4-orcamento.yaml`
Chat	`e2e/fase5-chat.yaml`

Os fluxos Maestro precisam do app instalado num emulador Android e ainda não foram executados neste ambiente (sem emulador). Cada fase também foi verificada de ponta a ponta na versão web do app, com o backend e o banco reais; os prints estão em `docs/telas/`.

Registro de decisões

Todas as escolhas feitas durante o desenvolvimento, principalmente onde o plano estava em aberto ou onde a implementação difere do Figma. Numeradas em ordem; itens atualizados em fases seguintes dizem isso no próprio texto.

Fase 0

#	Decisão	Motivo
1	O design vem de `docs/design/00-visao-geral.png`: uma imagem única com as 12 telas, já que não há link do Figma.	O usuário só tem a imagem. Detalhes finos (medidas exatas, hex exato) foram estimados a partir dela. Exportar cada frame em PNG para `docs/design/` melhora a fidelidade nas fases seguintes.
2	Tokens: primária `#5A1F3C`, fundo `#F8F2EC`, cards brancos com borda `#E8DDD4`, estrelas `#D9A035`.	Cores aproximadas a partir da imagem. Ajustar em `mobile/src/theme/tokens.ts` quando houver os valores exatos do Figma.
3	Fontes: **Playfair Display** (títulos, serifada) e **Inter** (corpo), via `@expo-google-fonts`.	O Figma usa uma serifada nos títulos e uma sans-serif no corpo; os nomes exatos não aparecem na imagem. Basta trocar em `theme/fonts.ts` e `theme/tokens.ts`.
4	Rotas do Expo Router em `mobile/src/app/` (e não `mobile/app/`, como diz o plano).	É a convenção do template do Expo SDK 58. A organização interna (`(auth)`, `(cliente)`, `(empresa)`, `chat/`) continua a mesma do plano.
5	Spring Boot **3.5.16** (último 3.x), e não a 4.x.	O plano pede Spring Boot 3.x.
6	Na Fase 0, `/` mostra o catálogo de componentes.	É o critério de aceite da Fase 0. Na Fase 1 vira a tela de boas-vindas.
7	A migration `V1__extensoes.sql` só habilita `unaccent`. As tabelas começam na Fase 1 (V2 em diante).	Fazer o Flyway rodar desde a Fase 0 sem antecipar o modelo; `unaccent` vai ser usado na busca da Fase 3.
8	Testes de integração com Testcontainers; se a variável `TEST_DB_URL` existir, usam esse banco.	Permite rodar em CI ou máquinas sem Docker.
9	Na web, a sessão fica só em memória (o SecureStore não existe lá).	A web serve só para visualizar a interface; o alvo é Android.
10	O componente `TabBar` só desenha a barra; é ligado ao navegador de abas a partir da Fase 1/2.	Mantém o componente testável e independente de rotas.

Fase 1

#	Decisão	Motivo
11	**Cadastro de cliente em 2 etapas:** etapa 1 igual ao Figma; etapa 2 com **CPF (opcional, sem validação)** e **foto (opcional)**.	Resposta do usuário: projeto simples, para apresentação.
12	**Cadastro de empresa em 2 etapas:** etapa 1 igual ao Figma; etapa 2 com senha, confirmação de senha, **endereço (opcional, sem verificação)** e aceite dos termos.	O Figma não tem campo de senha para a empresa; o endereço segue a resposta do usuário e pode ser editado depois no perfil (Fase 2).
13	O CNPJ **não confere os dígitos verificadores**; só exige 14 dígitos (com ou sem máscara) e continua único.	Pedido do usuário (projeto para apresentação). Os 14 dígitos são mantidos porque a máscara, o banco e o login por CNPJ dependem desse formato.

#	Decisão	Motivo
14	Os cadastros usam `multipart/form-data` com campos simples (não JSON).	Envia a foto/logo na mesma requisição e funciona bem com o `FormData` do React Native.
15	O cadastro não devolve token: o app volta ao login com "Cadastro realizado!" e o e-mail preenchido.	Pedido do plano.
16	Login na aba errada (senha certa) responde "Esta conta é de cliente. Entre pela aba Cliente.". Senha errada e usuário inexistente têm a **mesma** mensagem.	Mensagem clara sem revelar quais e-mails existem.
17	Cidade é texto livre (ex.: "São Paulo, SP"). No Figma ela tem uma seta de lista, mas não há lista de cidades definida. A UF é extraída quando vem depois da vírgula.	A opção mais simples; dá para trocar por um seletor depois.
18	As áreas logadas ficam em `src/app/cliente/` e `src/app/empresa/` (pastas comuns, não grupos com parênteses).	Com grupos, `(cliente)/(tabs)/index` e `(empresa)/(tabs)/index` teriam a mesma URL `/` (e o mesmo valia para `/perfil` e `/solicitacoes`), o que o Expo Router não permite.
19	O acesso às áreas é controlado com `Stack.Protected` no layout raiz: sem login mostra boas-vindas/login/cadastros; com login, só a área do perfil do usuário.	Redireciona sozinho após login e logout.
20	A foto da tela de boas-vindas foi substituída por um fundo bordô com "luzes" desfocadas.	Não temos a foto original. Basta exportar a imagem do Figma e trocar em `HeroBoasVindas.tsx`.
21	Uploads gravados em `backend/uploads/` e servidos em `/files/**`, atrás da interface `StorageService` (adiantado da Fase 2).	O logo e a foto já são enviados no cadastro.
22	O catálogo de componentes saiu da rota `/` e foi para `/catalogo`.	`/` agora é a tela de boas-vindas.

Fase 2

#	Decisão	Motivo
23	**Rascunho** exige só nome e categoria; **publicar** exige também descrição e preço conforme o modelo. Sem modelo escolhido, o rascunho fica como "Sob consulta".	O Figma tem "Salvar rascunho" e "Atualizar serviço"; o rascunho serve para salvar um serviço incompleto.
24	Um serviço só aparece para clientes se estiver **ativo e publicado**. A dona vê todos.	Combina a chave "ativo" do Figma com o rascunho.
25	Modelos de preço: "A partir de" (padrão, como no Figma), "Preço fixo", "Faixa de preço" (mín. e máx.) e "Sob consulta".	Seção 6 do plano.
26	Limites: 6 fotos por serviço (o Figma mostra "x de 6") e 12 no portfólio.	Evitar uploads sem fim; 12 é suficiente para um portfólio.
27	No formulário de serviço, as fotos novas e removidas ficam pendentes e só são enviadas ao salvar (depois de criar o serviço, porque o upload precisa do id). No **perfil**, logo, capa e portfólio são enviados **na hora**.	No serviço, permite cancelar a edição; no perfil, cada foto é uma ação independente.
28	E-mail e CNPJ não podem ser editados no perfil.	São as credenciais de login da empresa.
29	Excluir serviço apaga de vez (com as fotos) após uma confirmação. **Atualizado na Fase 4:** se o serviço aparece em alguma solicitação, a exclusão é bloqueada (409) e a empresa deve desativá-lo.	Preserva o histórico dos pedidos.

#	Decisão	Motivo
30	A tela de criar/editar serviço abre por cima das abas (sem a barra de abas), enquanto no Figma a barra aparece.	Os botões fixos "Salvar rascunho" / "Atualizar serviço" ficam mais visíveis e não competem com a barra.
31	O código da empresa aparece no perfil com o texto "Compartilhe com sua equipe".	Pensando no futuro perfil Funcionário (fora do escopo).
32	Na web, os uploads convertem a imagem em `Blob` antes de enviar (no Android o `FormData` aceita `{ uri, name, type }`).	Permite demonstrar o app pelo navegador.

Fase 3

#	Decisão	Motivo
33	**Notas e avaliações ficam para a Fase 6.** Até lá os cards e o perfil mostram o selo "Novo", e "Melhor avaliadas" ordena por relevância. O seed também não traz avaliações.	A tabela `avaliacao` depende de `solicitacao` (Fase 4): só quem concluiu um evento pode avaliar. Criar avaliações falsas agora quebraria essa regra.
34	Indicadores do perfil: **serviços publicados**, **fotos no portfólio** e **ano de entrada**, no lugar de "eventos realizados", "tempo de resposta" e "% recomendam" do Figma.	Esses três dependem de solicitações, chat e avaliações (Fases 4 a 6). O campo `eventosRealizados` já existe na API e passa a ser preenchido na Fase 4.
35	Busca com várias palavras: **todas** precisam aparecer (no nome, na categoria, num serviço ou nas descrições); palavras como "de", "para" e "e" são ignoradas.	"Decoração para casamento" deve trazer quem faz decoração **de** casamento, não qualquer decoração.
36	A busca usa SQL com `unaccent` em vez de JPA/Criteria.	Relevância ponderada, filtros opcionais e agregações (menor preço) ficam mais simples e rápidas em SQL.
37	Não implementados por estarem fora do escopo ou sem dados: **distância** ("4,2 km"), **favoritos** (coração), **mapa** e o **sino de notificações**. O filtro de faixa de preço do Figma ("até R$ 5.000") também não, porque o plano define só categoria, cidade e ordenação.	Exigiriam geolocalização, uma tabela nova ou push (fora do MVP).
38	Os filtros da busca ficam nos **parâmetros da rota** (`/cliente/buscar?q=&categoriaId=&cidade=&ordenar=`).	O Início abre a busca já filtrada (categoria, texto, banner) e o "voltar" preserva o estado.
39	Busca e recomendados são só para **clientes** (403 para empresa); o perfil público pode ser aberto por qualquer usuário logado, mas só a visita de cliente conta como visualização.	Seção 7 do plano.
40	Seed de demonstração em Java (`DemoSeeder`), com imagens geradas por código (gradiente com "luzes"), no lugar de fotos reais.	Evita fotos com direitos autorais e roda sem depender de internet.
41	"Solicitar orçamento" no perfil ainda mostra "Em breve".	O fluxo de orçamento é a Fase 4.

Fase 4

#	Decisão	Motivo
42	Novo status **CANCELADA**, para quando o cliente desiste antes de aceitar.	O plano diz que o cliente pode cancelar antes de ACEITA, mas o diagrama não tinha um estado para isso.
43	**Recusar a proposta** não encerra o pedido: a proposta fica RECUSADA e a solicitação continua EM_NEGOCIACAO, para a empresa mandar outra. Só uma proposta pode estar aguardando resposta por vez (índice único no banco).	Recusar costuma significar "quero outro valor"; para desistir de vez existe o cancelamento.

#	Decisão	Motivo
44	A empresa **não** passa para EM_NEGOCIACAO pelo PATCH de status: isso só acontece ao enviar uma proposta.	EM_NEGOCIACAO significa "proposta enviada" no plano.
45	A **mensagem do tipo PROPOSTA no chat** fica para a Fase 5 (a tabela `mensagem` nasce lá). A conversa já é criada junto com a solicitação. **Feito na Fase 5**, inclusive para propostas antigas (migration V6).	Segue a ordem das fases do plano.
46	Taxa de resposta = % das solicitações dos últimos 30 dias que saíram de PENDENTE, **sem contar as canceladas** pelo cliente. O período é fixo em 30 dias (o seletor "Últimos 30 dias" do Figma é só um rótulo).	Uma solicitação cancelada antes de a empresa abrir não deveria baixar a taxa. Seletor de período não está no plano.
47	Faixas de orçamento fixas (até 5 mil, 5–10, 10–20, 20–50, acima de 50 mil, "Prefiro não informar"), gravadas como mínimo e máximo.	O Figma mostra um seletor de faixa; guardar números permite filtros futuros.
48	O pedido tem 3 etapas: **dados → revisão → confirmação**, como a barra de 3 partes do Figma. Inspirações são enviadas depois de criar a solicitação (precisam do id).	Critério de aceite da fase.
49	"Responder", no painel e na lista da empresa, abre o detalhe já com o formulário de proposta. Contato direto pelo WhatsApp disponível nos dois lados.	O chat em tempo real é a Fase 5.
50	Listas, detalhe e painel sempre recarregam ao abrir a tela (`staleTime: 0`).	O status muda pelo outro lado; sem chat/push ainda, é o jeito simples de "o cliente ver a atualização".
51	Na web, o campo de data é texto com máscara dd/mm/aaaa (o seletor nativo não existe na web).	A web serve para demonstração.

Fase 5

#	Decisão	Motivo
52	STOMP sobre **WebSocket puro** em `/ws` (sem SockJS), com o broker simples do Spring em memória.	O React Native tem WebSocket nativo; o broker em memória basta para uma instância (para escalar, troca-se por RabbitMQ sem mudar o app).
53	Autenticação no **CONNECT** do STOMP (JWT no cabeçalho), e checagem de participação em cada SUBSCRIBE e SEND. O HTTP de `/ws` é público.	Navegadores e o RN não mandam cabeçalhos customizados no handshake HTTP do WebSocket.
54	As mensagens só são publicadas no tópico **depois do commit** da transação.	Evita o app receber uma mensagem que depois sofreu rollback.
55	Proposta aceita/recusada **republica a mesma mensagem** com o status novo; o app faz "upsert" pelo id. Não existe um tipo "SISTEMA".	O plano define só TEXTO e PROPOSTA; assim o card muda de status nos dois lados em tempo real.
56	O cliente pode **aceitar/recusar a proposta direto no card do chat** (com confirmação). "Ver proposta" abre o detalhe da solicitação.	Pedido do plano para a tela 9.
57	Histórico paginado por **cursor** (`antesDe` = id), 30 mensagens por vez. Lista invertida que carrega as mais antigas ao rolar para cima.	Cursor não "pula" mensagens quando chegam novas durante a rolagem.
58	A conversa é marcada como lida ao abrir e a cada mensagem recebida. A confirmação de leitura (✓ verde) do remetente atualiza quando ele reabre a conversa (não é empurrada em tempo real).	Mantém o protocolo simples; o estado no servidor está sempre correto.
59	A lista de conversas não tem aba própria: abre pelo **ícone de conversas** (com contador de não lidas) no Início do cliente e no Painel da empresa, e pelo botão "Conversar"/"Abrir conversa" nos detalhes da solicitação.	O Figma tem 4 abas por perfil e nenhuma é de conversas.
60	Não implementados: anexos no chat (botão "+"), mensagem de voz (microfone), indicador "digitando..." e status "online".	Fora do escopo do plano.

Pontos em aberto (aguardando o usuário)

1. ~~Cadastro de cliente, etapa 2~~ → decidido (decisão 11).
2. ~~Endereço da empresa~~ → decidido (decisão 12).
3. **Diagrama de classes (`APP_EVENTOS.pdf`):** não fornecido; o modelo segue a seção 6 do plano.
4. **Link do Figma:** não existe; usando a imagem (decisão 1).

Anexo: plano de implementação

> **Como usar este arquivo:** coloque-o na raiz de um repositório vazio (pode renomear para `CLAUDE.md` ou citar com `@SYNCEVENTS_PLANO.md`) e peça ao Claude Code: *"Leia o plano e execute a Fase 0. Ao final de cada fase, rode os testes, faça commit e pare para eu revisar."*

---

1. Contexto do produto

Item	Definição
Nome	SyncEvents
Tema	Facilitar o contato entre clientes e empresas organizadoras de eventos, centralizando informações sobre serviços e simplificando a comunicação entre as partes.
Problema	Clientes têm dificuldade para encontrar empresas adequadas para seus eventos e para falar com elas de forma organizada.
Objetivo	Uma solução que facilite a busca por empresas de eventos e a comunicação entre clientes e prestadores.
Público-alvo	Pessoas organizando eventos; empresas organizadoras de eventos; prestadores de serviços para eventos.
Plataforma	**Android** (app mobile)
Perfis de usuário	`CLIENTE` e `EMPRESA`

---

2. Stack (decidida)

O requisito é "React + Java para Android". A forma mais direta de cumprir isso é:

2.1 Mobile: React Native (Expo) + TypeScript

Função	Biblioteca
Framework	**Expo SDK (versão estável mais recente)** + React Native + **TypeScript (strict)**
Navegação	**Expo Router** (rotas por arquivo, grupos `(auth)`, `(cliente)`, `(empresa)`)
Dados do servidor / cache	**TanStack Query**
HTTP	**axios**, com interceptor que injeta o JWT e trata 401
Estado global (sessão)	**Zustand**
Armazenamento seguro do token	**expo-secure-store**
Formulários / validação	**react-hook-form** + **zod** (os mesmos schemas usados para as mensagens de erro)
Imagens (logo, portfólio, fotos do serviço)	**expo-image-picker** + `expo-image`
Chat em tempo real	**@stomp/stompjs** (WebSocket STOMP com o backend)
Datas	`@react-native-community/datetimepicker` + `date-fns` (locale pt-BR)
Fontes	`expo-font` (usar as fontes do Figma)

Função	Biblioteca
Testes	**Jest** + **React Native Testing Library**; E2E com **Maestro** (fluxos em YAML)
Qualidade	ESLint + Prettier

Build Android: `npx expo run:android` (dev) e **EAS Build** ou `expo prebuild` + Gradle para gerar o APK.

2.2 Backend: Java + Spring Boot

Função	Tecnologia
Linguagem / build	**Java 21**, **Maven**
Framework	**Spring Boot 3.x**: Web, Validation, Data JPA, Security, WebSocket
Autenticação	**JWT** (lib `jjwt`), senha com **BCrypt**
Banco	**PostgreSQL 16**
Migrations	**Flyway** (`src/main/resources/db/migration/V1__...sql`)
Documentação da API	**springdoc-openapi** (Swagger UI em `/swagger-ui.html`)
DTOs	Java `record` + mapeamento manual ou **MapStruct**
Upload de arquivos	Multipart → pasta local `uploads/`, servida em `/files/**` (trocar por S3/MinIO depois, atrás de uma interface `StorageService`)
Testes	JUnit 5, Mockito, **Testcontainers** (Postgres), MockMvc

2.3 Infra local

- `docker-compose.yml` na raiz com **postgres** (e opcionalmente **pgAdmin**).
- Backend roda em `http://localhost:8080`; no emulador Android a base URL é **`http://10.0.2.2:8080`** (configurar via `EXPO_PUBLIC_API_URL`).

---

3. Estrutura do repositório (monorepo)

```
syncevents/
├── CLAUDE.md                  # este plano (ou um resumo apontando para ele)
├── docker-compose.yml
├── docs/
│   ├── design/                # PNGs exportados do Figma (uma por tela)
│   ├── api.md                 # gerado/atualizado a cada fase
│   └── decisoes.md            # registro de decisões tomadas durante o desenvolvimento
├── backend/
│   ├── pom.xml
│   └── src/main/java/br/com/syncevents/
│       ├── config/            # Security, CORS, WebSocket, OpenAPI
│       ├── auth/              # login, JWT, registro
│       ├── usuario/           # entidade base Usuario
│       ├── cliente/
│       ├── empresa/           # perfil, portfólio, dashboard
│       ├── categoria/
│       ├── servico/
│       ├── busca/
│       ├── solicitacao/       # orçamentos + propostas
│       ├── chat/              # conversas e mensagens (REST + STOMP)
│       ├── avaliacao/
│       ├── storage/
│       └── common/            # exceptions, handler global, validadores (CNPJ, senha)
└── mobile/
    ├── app/                   # Expo Router
    │   ├── _layout.tsx
    │   ├── index.tsx                    # Boas-vindas
    │   ├── (auth)/login.tsx
    │   ├── (auth)/cadastro-cliente.tsx
    │   ├── (auth)/cadastro-empresa.tsx
    │   ├── (cliente)/(tabs)/...         # Início, Buscar, Solicitações, Perfil
    │   ├── (cliente)/empresa/[id].tsx
    │   ├── (cliente)/orcamento/[empresaId].tsx
    │   ├── (empresa)/(tabs)/...         # Painel, Solicitações, Serviços, Perfil
    │   ├── (empresa)/servico/[id].tsx   # criar/editar ("novo" = criação)
    │   └── chat/[conversaId].tsx        # compartilhado pelos dois perfis
    ├── src/
    │   ├── api/               # cliente axios + funções por recurso
    │   ├── hooks/             # hooks do TanStack Query
    │   ├── components/        # design system (Button, Input, Chip, Card, RatingStars...)
    │   ├── theme/             # tokens: cores, tipografia, espaçamento, raios
    │   ├── store/             # zustand (sessão)
    │   ├── schemas/           # zod
    │   └── utils/             # máscaras (telefone, CNPJ, moeda), formatação
    └── e2e/                   # fluxos Maestro
```

---

4. Design (Figma)

As telas já estão desenhadas no Figma. **A UI deve seguir o Figma fielmente** (layout, textos, espaçamentos, ícones).

Como o Claude Code deve obter o design (nesta ordem):

1. Se houver um **servidor MCP do Figma** configurado, use-o para ler frames, variáveis de cor e tipografia. Link do arquivo: `<<COLAR AQUI O LINK DO FIGMA>>`
2. Senão, use os PNGs em `docs/design/` (exporte cada frame do Figma com o nome da tela, ex.: `07-perfil-da-empresa.png`).
3. Se nenhum dos dois existir, **pare e peça** ao usuário antes de inventar layout.

Identidade visual (aproximada, CONFIRMAR no Figma antes de fixar os tokens):

- Primária: vinho / bordô (~`#5A1F3C`); usada em botões, chips selecionados, tab ativa
- Fundo: creme / off-white (~`#F8F2EC`); cards em branco com borda suave
- Títulos em **fonte serifada**; corpo em **sans-serif**
- Cantos bem arredondados (botões em formato pílula), chips arredondados
- Estrelas de avaliação + nota (ex.: 4,9)

Crie `mobile/src/theme/tokens.ts` com esses valores e **nunca use cores fixas nos componentes**.

4.1 Mapa de telas (Figma ↠ rota ↠ história)

#	Tela no Figma	Rota	Perfil	História
1	Tela de boas-vindas (escolha "Sou cliente" / "Sou empresa" + "Continuar")	`app/index.tsx`	público	—
2	Login (abas Cliente/Empresa, e-mail, senha, "Esqueci minha senha", Entrar, "Continuar com Google", link Cadastre-se)	`(auth)/login`	público	ID03
3	Cadastro de cliente ("Etapa 1 de 2"; nome completo, e-mail, celular, cidade, senha, chips de tipo de evento de interesse, aceite dos termos)	`(auth)/cadastro-cliente`	público	ID01
4	Cadastro de empresa (logo, nome fantasia, CNPJ, e-mail comercial, telefone, cidade, categorias principais em chips, "Sobre a empresa")	`(auth)/cadastro-empresa`	público	ID02
5	Cliente — Início ("Vamos celebrar?", busca, "Explore por categoria", banner, "Recomendados para você")	`(cliente)/(tabs)/index`	cliente	ID06
6	Resultados de busca (contagem + cidade, filtros em chips, cards com categoria, nota e "a partir de R$")	`(cliente)/(tabs)/buscar`	cliente	ID06
7	Perfil da empresa (capa, logo, nome, categoria, cidade, nota/nº de avaliações, indicadores, Portfólio, Serviços, Avaliações, botão "Solicitar orçamento")	`(cliente)/empresa/[id]`	cliente	ID07
8	Solicitação de orçamento (data, nº de convidados, local, tipo de evento, serviços desejados, faixa de orçamento, mensagem, anexar inspirações, aceite de contato, "Revisar solicitação")	`(cliente)/orcamento/[empresaId]`	cliente	ID08
9	Conversa com empresa (cabeçalho com o orçamento e status "Em negociação", mensagens, card de "Proposta inicial" com valor e "Ver proposta")	`chat/[conversaId]`	ambos	ID09
10	Avaliação de empresa ("Minhas solicitações" com abas Todas/Aguardando/Propostas/Concluídas + bottom sheet "Evento concluído" com estrelas, chips de destaque, comentário, "Publicar avaliação")	`(cliente)/(tabs)/solicitacoes`	cliente	ID11
11	Painel da empresa ("Olá, {empresa}", visão geral com visualizações, solicitações e taxa de resposta, lista "Solicitações recebidas" com "Ver detalhes"/"Responder")	`(empresa)/(tabs)/index`	empresa	ID10 / ID04
12	Cadastro/Editar serviço (serviço ativo, fotos, nome, categoria, descrição, modelo de preço, valor, itens inclusos, "Salvar rascunho"/"Atualizar serviço")	`(empresa)/servico/[id]`	empresa	ID05

Telas **sem frame no Figma** que precisam existir (fazer no mesmo estilo, de forma simples): editar perfil da empresa (ID04), lista de serviços da empresa, detalhe de uma solicitação (empresa), lista de conversas, perfil do cliente / logout.

---

5. Regras de negócio (da documentação do Trello)

5.1 Validações

- **Senha** (cliente e empresa): mín. 8 caracteres, com **maiúscula, minúscula, número e caractere especial**. Validar no app (zod) **e** no backend (validador customizado).
- **E-mail**: formato válido e único no sistema.
- **CNPJ**: 14 dígitos com **validação dos dígitos verificadores**, único. Máscara `00.000.000/0000-00`.
- **Telefone**: máscara `(00) 00000-0000`. O telefone da empresa é o contato de **WhatsApp** (exibir botão que abre `https://wa.me/55<numero>` no perfil).
- **Nome** (pessoa): apenas letras (com acento) e espaços.
- **Código da empresa**: ao cadastrar, gerar um **código público de 6 dígitos aleatórios e único** (`codigo`), exibido no perfil da empresa. (Fica pronto para o futuro perfil "Funcionário", que é fora do escopo.)

5.2 Categorias de serviço

Tabela `categoria` com seed: **Decoração, Buffet, Fotografia, Som e luz, Planejamento, Bebidas** (as quatro primeiras são as do Figma; Planejamento e Bebidas vêm do campo "Especialidade" da documentação). Cada categoria tem um ícone.

5.3 Tipos de evento

Enum: `CASAMENTO, ANIVERSARIO, CORPORATIVO, FORMATURA, INFANTIL, OUTRO`.

5.4 Status da solicitação de orçamento

```
PENDENTE ──► EM_ANALISE ──► EM_NEGOCIACAO (proposta enviada) ──► ACEITA ──► CONCLUIDA
     └──────────────┴──────────────┴──► RECUSADA        (cliente pode CANCELAR antes de ACEITA)
```

- Abas do cliente: **Aguardando** = PENDENTE/EM_ANALISE · **Propostas** = EM_NEGOCIACAO/ACEITA · **Concluídas** = CONCLUIDA.
- A empresa altera o status; o cliente aceita/recusa a proposta.
- Ao criar a solicitação, **criar automaticamente a conversa** entre cliente e empresa, ligada à solicitação.

5.5 Avaliação

- Só quem tem uma solicitação **CONCLUIDA** com a empresa pode avaliar, **uma avaliação por solicitação**.
- Nota **1 a 5**, comentário opcional, destaques opcionais (`PONTUALIDADE, QUALIDADE, ATENDIMENTO, CUSTO_BENEFICIO`).
- O perfil da empresa mostra a **média** (1 casa decimal, formato pt-BR "4,9") e o total de avaliações.

5.6 Indicadores do painel da empresa

- **Visualizações do perfil**: contar cada abertura do perfil por um cliente (tabela `visualizacao_perfil`), comparar com os 30 dias anteriores (%).
- **Solicitações**: total no período.
- **Taxa de resposta**: % de solicitações que saíram de PENDENTE.

---

6. Modelo de dados (PostgreSQL)

> O Trello tem um **Diagrama de Classes** (`APP_EVENTOS.pdf`) anexado que não veio no JSON. Se o usuário colocar esse PDF em `docs/`, **compare com o modelo abaixo e alinhe os nomes** antes de criar as migrations.

```
usuario(id PK, email UNIQUE, senha_hash, tipo [CLIENTE|EMPRESA], ativo, criado_em)
cliente(id PK, usuario_id FK UNIQUE, nome, telefone, cidade, foto_url NULL)
cliente_interesse(cliente_id FK, tipo_evento)                    -- chips do cadastro
empresa(id PK, usuario_id FK UNIQUE, codigo CHAR(6) UNIQUE, nome_fantasia, cnpj UNIQUE,
        telefone_whatsapp, cidade, uf, endereco, descricao, logo_url, capa_url, criado_em)
empresa_categoria(empresa_id FK, categoria_id FK)
categoria(id PK, nome UNIQUE, icone)
portfolio_foto(id PK, empresa_id FK, url, ordem)
servico(id PK, empresa_id FK, categoria_id FK, nome, descricao,
        modelo_preco [FIXO|A_PARTIR_DE|FAIXA|SOB_CONSULTA], preco_min NUMERIC NULL, preco_max NUMERIC NULL,
        ativo, rascunho, criado_em, atualizado_em)
servico_foto(id PK, servico_id FK, url, ordem)
servico_item_incluso(id PK, servico_id FK, descricao)
solicitacao(id PK, cliente_id FK, empresa_id FK, tipo_evento, data_evento, qtd_convidados, local,
           orcamento_min NULL, orcamento_max NULL, mensagem, status, aceite_contato, criado_em, atualizado_em)
solicitacao_servico(solicitacao_id FK, servico_id FK)            -- "serviços desejados"
solicitacao_anexo(id PK, solicitacao_id FK, url)                 -- "inspirações"
proposta(id PK, solicitacao_id FK, valor NUMERIC, descricao, status [ENVIADA|ACEITA|RECUSADA], criado_em)
conversa(id PK, solicitacao_id FK UNIQUE, cliente_id FK, empresa_id FK, criado_em)
mensagem(id PK, conversa_id FK, remetente_usuario_id FK, tipo [TEXTO|PROPOSTA], conteudo, proposta_id NULL,
lida, enviada_em)
avaliacao(id PK, solicitacao_id FK UNIQUE, empresa_id FK, cliente_id FK, nota SMALLINT CHECK 1..5,
          comentario NULL, destaques TEXT[] , criado_em)
visualizacao_perfil(id PK, empresa_id FK, cliente_id FK NULL, criado_em)
```

Valores monetários em `NUMERIC(12,2)` / `BigDecimal`; no app formatar como `R$ 18.500`.

---

7. API REST (contrato)

Prefixo `/api/v1`. Todas as rotas, exceto `auth` e `GET` públicos de categoria, exigem `Authorization: Bearer <jwt>`. Erros no formato:

```
{ "status": 400, "erro": "VALIDACAO", "mensagem": "Dados inválidos", "campos": { "email": "E-mail já
cadastrado" } }
```

Método	Rota	Perfil	Descrição
POST	`/auth/cadastro/cliente`	público	ID01
POST	`/auth/cadastro/empresa`	público	ID02 (multipart com logo opcional)
POST	`/auth/login`	público	ID03: `{ login, senha, tipo }`; `login` aceita e-mail ou CNPJ quando `tipo=EMPRESA` → `{ token, usuario }`
GET	`/auth/me`	ambos	dados do usuário logado
GET	`/categorias`	público	lista
GET	`/empresas?q=&categoriaId=&cidade=&page=&size=&ordenar=relevancia|nota|preco`	cliente	ID06 (busca por nome, nome de serviço ou categoria)

Método	Rota	Perfil	Descrição
GET	`/empresas/recomendadas`	cliente	Início: melhor nota, filtrado pelos interesses/cidade do cliente
GET	`/empresas/{id}`	ambos	ID07: perfil completo (registra visualização se for cliente)
GET/PUT	`/empresas/me`	empresa	ID04: ver/editar o próprio perfil
POST/DELETE	`/empresas/me/portfolio`	empresa	fotos do portfólio
GET	`/empresas/me/dashboard`	empresa	indicadores do painel
GET	`/empresas/{id}/servicos`	ambos	serviços ativos
GET/POST	`/servicos/me`	empresa	ID05: listar/criar
PUT/DELETE	`/servicos/{id}`	empresa (dona)	editar/remover
POST	`/servicos/{id}/fotos`	empresa (dona)	upload
POST	`/solicitacoes`	cliente	ID08
GET	`/solicitacoes/me?status=`	ambos	cliente: as suas; empresa: recebidas (ID10)
GET	`/solicitacoes/{id}`	envolvidos	detalhe
PATCH	`/solicitacoes/{id}/status`	empresa	ID10: EM_ANALISE / RECUSADA / CONCLUIDA
POST	`/solicitacoes/{id}/propostas`	empresa	envia proposta (vira mensagem do tipo PROPOSTA no chat e muda status para EM_NEGOCIACAO)
PATCH	`/propostas/{id}`	cliente	aceitar/recusar
PATCH	`/solicitacoes/{id}/cancelar`	cliente	cancelar
GET	`/conversas`	ambos	ID09: lista com última mensagem e não lidas
GET	`/conversas/{id}/mensagens?antesDe=&size=`	envolvidos	histórico paginado
POST	`/conversas/{id}/mensagens`	envolvidos	envia (fallback REST)
WS	`/ws` (STOMP) → envia `/app/conversas/{id}`, assina `/topic/conversas/{id}`	envolvidos	tempo real, JWT no header `CONNECT`
POST	`/avaliacoes`	cliente	ID11

Método	Rota	Perfil	Descrição
GET	`/empresas/{id}/avaliacoes?page=`	ambos	lista + média

Toda rota que acessa um recurso de outra pessoa deve **checar a posse/participação** (retornar 403/404 caso contrário).

---

8. Ordem de implementação (fases)

Cada fase entrega **backend + mobile + testes** daquela funcionalidade. Ao final de cada fase: `mvn verify` e `npm test` passando, commit (`feat(fase-N): ...`), atualização de `docs/api.md` e uma **pausa para revisão do usuário**.

Fase 0: Fundação

- Monorepo, `docker-compose.yml`, `.gitignore`, `.env.example` (backend e mobile), README com como rodar.
- Backend: projeto Spring Boot, perfil `dev`, Flyway rodando, handler global de erros, OpenAPI, CORS, endpoint `/actuator/health`.
- Mobile: Expo + TypeScript + Expo Router, `theme/tokens.ts`, fontes, cliente axios com `EXPO_PUBLIC_API_URL`, TanStack Query provider, ESLint/Prettier.
- **Design system** base em `src/components/`: `Button` (primário/secundário/pílula), `TextInput` (com label, erro, máscara, ícone de olho na senha), `Chip` (selecionável), `Card`, `RatingStars`, `Avatar`, `SectionHeader`, `EmptyState`, `TabBar` estilizada.
- **Aceite:** app abre no emulador mostrando uma tela de catálogo dos componentes; backend sobe com o banco do docker.

Fase 1 (Sprint 1): Autenticação e cadastros (ID01, ID02, ID03)

- Migrations de `usuario`, `cliente`, `cliente_interesse`, `empresa`, `categoria` (+ seed), `empresa_categoria`.
- Spring Security stateless + JWT; validadores de senha e CNPJ; geração do `codigo` de 6 dígitos.
- Telas 1 a 4 do Figma. Após o cadastro: **voltar para o Login** com mensagem de sucesso (como diz o tutorial). Após o login: redirecionar para `(cliente)` ou `(empresa)` conforme o tipo. Sessão persistida no SecureStore; logout.
- "Continuar com Google" e "Esqueci minha senha": **visíveis, mas mostram "Em breve"** (fora do MVP).
- **Aceite:** e-mail duplicado, CNPJ inválido, senha fraca e login errado mostram mensagens claras; testes unitários dos validadores; testes de integração dos 3 endpoints; fluxo Maestro "cadastrar cliente → logar".

Fase 2 (Sprint 2): Empresa: perfil e serviços (ID04, ID05)

- Migrations de `servico`, fotos, itens inclusos, `portfolio_foto`; `StorageService` local.
- Tela 12 (criar/editar serviço, com rascunho e ativo/inativo), lista "Meus serviços" com editar/excluir (confirmação antes de excluir).
- Ver e editar o próprio perfil (logo, capa, descrição, contatos, cidade, categorias, portfólio).
- Tab bar da empresa com Painel (placeholder até a Fase 4), Solicitações, Serviços, Perfil.
- **Aceite:** empresa cria, edita, desativa e remove serviço; outra empresa não consegue editar (teste de 403).

Fase 3 (Sprint 2): Cliente: descoberta (ID06, ID07)

- Busca com filtro por categoria, cidade e ordenação; paginação com scroll infinito.
- Tela 5 (Início com categorias, banner e recomendados), tela 6 (resultados) e tela 7 (perfil da empresa com portfólio, serviços, avaliações, indicadores e botão de WhatsApp).
- Registrar a visualização do perfil.
- Gerar um **seed de dados de demonstração** (perfil `dev`): cerca de 10 empresas com serviços, fotos e avaliações, para as telas não ficarem vazias.

- **Aceite:** buscar "decoração" retorna as empresas da categoria e as que têm serviço com esse nome; estados de carregando, vazio e erro em todas as listas.

Fase 4 (Sprint 3): Orçamentos e gestão (ID08, ID10)

- Migrations de `solicitacao`, `solicitacao_servico`, `solicitacao_anexo`, `proposta`, `conversa`.
- Tela 8 com passo de **revisão** antes de enviar e tela de confirmação de envio.
- Empresa: lista de solicitações recebidas, detalhe, mudar status, enviar proposta.
- Tela 11 (painel da empresa) com os indicadores reais.
- Cliente: aba "Minhas solicitações" (parte de lista da tela 10) com as abas por status.
- **Aceite:** máquina de estados testada (transições inválidas retornam 409); o cliente vê a atualização do status.

Fase 5 (Sprint 3): Chat (ID09)

- Migration de `mensagem`; STOMP sobre WebSocket com autenticação JWT; endpoints REST de histórico.
- Lista de conversas + tela 9 (cabeçalho com o contexto do orçamento, bolhas, card de proposta com aceitar/recusar para o cliente, marcar como lida).
- Reconexão automática; se o WebSocket cair, fazer polling a cada 5 s.
- **Aceite:** duas sessões (cliente e empresa) trocam mensagens em tempo real; o histórico carrega paginado; usuário de fora recebe 403.

Fase 6 (Sprint 3): Avaliações (ID11)

- Migration de `avaliacao`; regra "só CONCLUIDA, uma por solicitação".
- Bottom sheet da tela 10; média e lista no perfil da empresa (tela 7).
- **Aceite:** não é possível avaliar duas vezes nem avaliar sem ter concluído (teste); a média é atualizada no perfil.

Fase 7: Acabamento e entrega

- Revisão visual tela a tela contra o Figma (listar as diferenças em `docs/decisoes.md`).
- Acessibilidade: `accessibilityLabel` nos botões de ícone, área de toque ≥ 48dp, contraste.
- Fluxos Maestro completos: cliente (cadastro → busca → orçamento → chat → avaliação) e empresa (cadastro → serviço → responder → proposta).
- Gerar o **APK** de release e documentar no README.
- Preencher o cartão "Documentação de tudo que foi feito" com o resumo de `docs/`.

---

9. Convenções para o Claude Code

- **Idioma:** código (classes, variáveis, rotas) em **português** seguindo os nomes deste plano (ex.: `Solicitacao`, `/solicitacoes`); textos da UI em pt-BR, iguais aos do Figma.
- **Backend:** organizar por funcionalidade (pacote por domínio) com `Controller → Service → Repository`; nada de lógica no controller; DTOs de entrada e saída separados das entidades; `@Transactional` no service.
- **Mobile:** telas finas; chamadas de API só pelos hooks em `src/hooks`; cores, espaçamentos e fontes só pelos tokens.
- **Testes obrigatórios por fase:** service (unitário), controller (MockMvc), regras de autorização e, no mobile, validação dos formulários e componentes principais.
- **Commits:** Conventional Commits, um por fase (ou mais, se a fase for grande).
- **Na dúvida:** não invente regra de negócio. Registre a dúvida em `docs/decisoes.md`, escolha a opção mais simples e avise na revisão da fase.
- **Fora do escopo do MVP:** perfil Funcionário, login com Google, recuperação de senha, pagamentos, notificações push. Deixe pontos de extensão, mas não implemente.

---

10. Pontos em aberto (o Claude Code deve perguntar ou registrar em `docs/decisoes.md`)

1. **"Etapa 1 de 2" no cadastro de cliente:** o Figma só mostra a etapa 1. O tutorial do ID01 cita **CPF**, que não aparece no design. Proposta: a etapa 2 coleta CPF (opcional) e foto. Confirmar com o usuário.
2. **Endereço da empresa:** o tutorial do ID02 pede endereço, o Figma pede só cidade. Proposta: cidade obrigatória e endereço opcional, editável no perfil.
3. **Diagrama de classes (`APP_EVENTOS.pdf`):** não está no JSON; alinhar o modelo da seção 6 se o PDF for fornecido.
4. **Link do Figma:** preencher na seção 4.